

UNIT I

Digital System And Binary Numbers: Number System and its arithmetic Signed binary numbers, Logic simplification and combinational logic design: Binary codes, code conversion, review of Boolean algebra and Demorgans theorem, SOP & POS forms, Canonical forms, Karnaugh maps method up to five variable, Don't care conditions, POS simplification, NAND and NOR implementation, Quine McClusky method (Tabular method).

1 Number System and Its Arithmetic

1.1 Introduction to Number Systems

A number system is a way to represent numbers using symbols and rules.

Every number system has:

- **Base (Radix)** → total digits used
- **Place value** → positional weight

General representation:

$$N = d_n r^n + d_{n-1} r^{n-1} + \dots + d_0 r^0$$

Where:

- r = base
- d = digit

1.2 Types of Number Systems

(A) Decimal (Base 10)

Digits: 0–9

Used in daily life.

Example:

$$(345)_{10} = 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$$

(B) Binary (Base 2) ★ MOST IMPORTANT

Digits: 0 and 1

Why binary?

Because electronic circuits have only 2 stable states:

- 0 → LOW voltage
- 1 → HIGH voltage

So CPU, RAM, ALU all work in binary.

Example:

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ = 8 + 0 + 2 + 1 = 11$$

(C) Octal (Base 8)

Digits: 0–7

Shortcut representation of binary.

1 octal digit = 3 binary bits.

(D) Hexadecimal (Base 16)

Digits:

0–9 and A–F

A = 10

B = 11

C = 12

D = 13

E = 14

F = 15

1 hex digit = 4 binary bits.

Used in:

- Memory addresses
- Machine code
- Debugging

Example:

$$(2F)_{16} \\ = 2 \times 16 + 15 \\ = 47$$

1.3 Number Conversion

Decimal → Binary

Method: Repeated division by 2.

Example: Convert 25 to binary.

$$25 \div 2 = 12 \text{ r}1$$

$$12 \div 2 = 6 \text{ r}0$$

$$6 \div 2 = 3 \text{ r}0$$

$$3 \div 2 = 1 \text{ r}1$$

$$1 \div 2 = 0 \text{ r}1$$

Write remainder bottom-up:

$$(25)_{10} = (11001)_2$$

Binary → Decimal

Multiply each bit with power of 2.

Example:

$$(10101)_2$$

$$= 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$$= 21$$

Binary ↔ Octal

Group into 3 bits.

Example:

$$110101_2$$

$$= 110 \ 101$$

$$= 6 \ 5$$

$$= (65)_8$$

Binary ↔ Hexadecimal

Group into 4 bits.

Example:

$$11101101$$

$$= 1110 \ 1101$$

$$= E \ D$$

$$= (ED)_{16}$$

1.4 Binary Arithmetic

Binary Addition

Rules:

$$0+0 = 0$$

$$0+1 = 1$$

$$1+0 = 1$$

$$1+1 = 0 \text{ carry } 1$$

Example:

1011

- 1101

11000

Binary Subtraction

Better method: Use 2's complement.

Binary Multiplication

Example:

101

× 11

101

101

1111

2 Signed Binary Numbers

Computers must store negative numbers.

2.1 Sign-Magnitude

Leftmost bit = sign

0 → positive

1 → negative

Example (4-bit):

+6 = 0110

-6 = 1110

Problem:

Two zeros exist → +0 and -0

2.2 1's Complement

Flip bits.

+6 = 0110

-6 = 1001

Again two zeros problem.

2.3 2's Complement ★ VERY IMPORTANT

Steps:

1. Take 1's complement
2. Add 1

Example:

Find -6 (4-bit)

6 = 0110

1's complement = 1001

Add 1 = 1010

So -6 = 1010

Subtraction Using 2's Complement

Example:

9 - 5

9 = 1001

5 = 0101

Find 2's complement of 5:

0101 $\rightarrow$ 1010 + 1 = 1011

Add:

1001

+1011

10100

Ignore carry $\rightarrow$ 0100 = 4

Correct.

3 Binary Codes

Binary codes represent numbers or characters in digital systems.

3.1 BCD (Binary Coded Decimal)

Each decimal digit = 4-bit binary.

Example:

59

5 $\rightarrow$ 0101

9 $\rightarrow$ 1001

BCD = 0101 1001

Important:

BCD is not equal to pure binary.

3.2 Excess-3 Code

Add 3 to each decimal digit, then convert to binary.

Example:

$4 \rightarrow 4+3 = 7 \rightarrow 0111$

3.3 Gray Code

Property:

Only one bit changes between consecutive numbers.

Used in:

- Rotary encoders
- Robotics
- Error reduction

Binary $\rightarrow$ Gray rule:

First bit same

Next bit = XOR of current and previous binary bit

Example:

Binary: 1011

Gray:

1

$$1 \oplus 0 = 1$$

$$0 \oplus 1 = 1$$

$$1 \oplus 1 = 0$$

Result = 1110

4 Review of Boolean Algebra

Boolean algebra deals with variables that have only two values:

0 or 1.

4.1 Basic Operations

AND ($\cdot$)

OR ($+$)

NOT ($'$)

4.2 Important Laws

Identity Law

$$A + 0 = A$$

$$A \cdot 1 = A$$

Null Law

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

Complement Law

$$A + A' = 1$$

$$A \cdot A' = 0$$

Idempotent Law

$$A + A = A$$

$$A \cdot A = A$$

Absorption Law

$$A + AB = A$$

$$A(A + B) = A$$

Distributive Law

$$A(B + C) = AB + AC$$

5 DeMorgan's Theorem

Very important for implementation.

$$(A + B)' = A'B'$$

$$(AB)' = A' + B'$$

Used to convert:

- OR to NAND
- AND to NOR

6 SOP and POS Forms

SOP (Sum of Products)

Form:

Product terms added.

Example:

$$F = A'B + AB'$$

POS (Product of Sums)

Form:

Sum terms multiplied.

Example:

$$F = (A + B)(A' + C)$$

7 Canonical Forms

Minterm

Each term contains all variables.

Example (A,B,C):

$$m_0 = A'B'C'$$

$$m_1 = A'B'C$$

$$m_2 = A'BC'$$

...

Written as:

$$F = \Sigma(1,3,5)$$

Maxterm

Each term contains all variables in sum form.

Written as:

$$F = \Pi(0,2,4)$$

8 Karnaugh Map (K-Map)

Used to simplify Boolean expressions.

Rules:

1. Group 1's
2. Group must be power of 2
3. Make largest possible group
4. Groups can overlap
5. Use don't care if helpful

4 Variable K-map

16 cells.

Wrap-around allowed.

Don't Care Conditions

Marked as X.

Used when output is irrelevant for some input combinations.

We can treat X as 1 or 0 to simplify.

9 POS Simplification Using K-map

Instead of grouping 1's,

Group 0's.

Result gives simplified POS.

10 NAND and NOR Implementation

NAND and NOR are universal gates.

Any Boolean function can be implemented using only NAND or only NOR.

Using DeMorgan's theorem:

$$A + B = (A'B')'$$

So OR can be implemented using NAND.

1 1 Quine McCluskey Method (Tabular Method)

Used when variables > 4.

Algorithmic method.

Steps:

1. List minterms in binary
2. Group by number of 1's
3. Compare adjacent groups
4. Combine terms differing in one bit
5. Remove duplicates
6. Identify Prime Implicants
7. Select Essential Prime Implicants

Used in:

- Logic minimization software
- Digital circuit design automation

1 Binary vs Decimal Number System

Feature	Decimal	Binary
Base	10	2
Digits Used	0–9	0,1
Used In	Human calculations	Computers, digital circuits
Position Weight	Powers of 10	Powers of 2
Example	$(25)_{10}$	$(11001)_2$
Hardware Friendly	✗ No	✓ Yes

👉 In computers, everything (images, ML models, code) is stored in binary.

2 1's Complement vs 2's Complement

Feature	1's Complement	2's Complement
How to Find	Flip bits	Flip bits + add 1
Zero Representation	Two zeros (+0, -0)	Only one zero
Arithmetic Complexity	Slightly complex	Easy for hardware
Used In	Rarely used	Used in CPU, ALU
Subtraction Method	End-around carry needed	No extra step
Example (-5 for 4-bit)	1010	1011

★ Exam Tip: Always mention **2's complement is used in computers**.

3 BCD vs Pure Binary

Feature	BCD	Pure Binary
Conversion	Each decimal digit → 4 bits	Whole number converted
Efficiency	Less efficient	More efficient
Example (9)	1001	1001
Example (12)	0001 0010	1100
Storage Requirement	More bits	Fewer bits
Used In	Digital clocks, calculators	CPU operations

Common mistake: Students think BCD = binary. ✗ Not same.

4 Gray Code vs Binary Code

Feature	Binary Code	Gray Code
Bit Change Between Consecutive Numbers	Multiple bits may change	Only one bit changes
Error Chance	Higher	Lower
Used In	Arithmetic operations	Rotary encoders, robotics
Conversion	Direct	Needs XOR logic
Hardware Noise Resistance	Low	High

Why important?

In mechanical sensors, if multiple bits change together, error chance increases.

5 SOP vs POS

Feature	SOP (Sum of Products)	POS (Product of Sums)
Structure	AND terms added	OR terms multiplied
Based On	Minterms	Maxterms
K-map Grouping	Group 1's	Group 0's
Expression Form	$A'B + AB'$	$(A + B)(A' + C)$
Canonical Notation	Σ (minterms)	Π (maxterms)
Implementation	AND-OR logic	OR-AND logic

Exam trick:

If question says Σ , it is SOP.

If question says Π , it is POS.

6 Minterm vs Maxterm

Feature	Minterm	Maxterm
Based On	SOP	POS
Output Value	1	0
Contains	All variables in product form	All variables in sum form
Representation	$\Sigma()$	$\Pi()$
K-map	Mark 1's	Mark 0's

7 NAND vs NOR Gate

Feature	NAND	NOR
Definition	NOT(AND)	NOT(OR)
Symbol	$(AB)'$	$(A+B)'$
Universal Gate	Yes	Yes
Easier For	SOP implementation	POS implementation
Output 0 Condition	When both inputs = 1	When any input = 1

Both are universal gates.

That means any Boolean function can be implemented using only one type.

8 K-map vs Quine-McCluskey

Feature	K-map	Quine-McCluskey
Type	Graphical	Tabular algorithm
Suitable For	≤ 4 or 5 variables	> 4 variables
Easy To Solve	Yes	Time-consuming
Used In	Manual simplification	Software tools
Error Chance	Human error possible	Algorithm-based

In real industry $\rightarrow$ software uses Quine-McCluskey.

9 Sign-Magnitude vs 2's Complement

Feature	Sign-Magnitude	2's Complement
Sign Bit	MSB	MSB
Negative Number	Sign bit 1	2's complement
Zero Representation	Two zeros	One zero
Arithmetic	Difficult	Easy
Used In	Rare	All modern CPUs

10 AND vs OR Operation

Feature	AND	OR
Symbol	$\cdot$	$+$
Output 1 Condition	Both inputs 1	At least one input 1
Logic Meaning	Multiplication	Addition
Real-life Analogy	Both conditions true	Any one condition true